

8+1 CLOUD RUN SVC + JOB	5 FLEET MEMBERS	12 LEAST-PRIV IDENTITIES	6 THROTTLED QUEUES	107 RULES ASSERTIONS
--------------------------------------	------------------------------	---------------------------------------	---------------------------------	-----------------------------------



Deployment topology — what is running, under which identity, with which grant

Every number on this page is checkable with one command, which is the point: an architecture claim that cannot be falsified is marketing. **One job per deployable**, **one identity per deployable**, and the narrowest role that lets that identity do its job.

8

RUN SERVICES

1

RUN JOB

12

SERVICE ACCOUNTS

0

KEY FILES ANYWHERE

2.1 THE LIVE INVENTORY, AND HOW TO FALSIFY IT		
RESOURCE	N	VERIFY
Cloud Run services	8	<code>gcloud run services list --region us-central1</code>
Cloud Run jobs	1	<code>gcloud run jobs list</code>
Service accounts	12	<code>gcloud iam service-accounts list</code>
Cloud Tasks queues	6	<code>gcloud tasks queues list --location us-central1</code>
GCS buckets (app)	3	<code>gcloud storage ls</code>
Eventarc triggers	1	<code>gcloud eventarc triggers list</code>
Cloud Scheduler jobs	3	<code>gcloud scheduler jobs list</code>
Firestore composite indexes	11	<code>firestore.indexes.json</code>
Firestore vector index	1	<code>faces.embedding · 512-d · flat · cosine</code>
Rules assertions, 8 personas	107	<code>make rules-test</code> (no cloud creds)
Golden-fixture eval checks	169	<code>make eval</code> (25 fixtures)
Agent Runtime deployments	0	deliberate — see 4.5

2.2 DELIBERATELY NOT USED		knowing why is part of the design
GKE	Identical container semantics to Cloud Run with a cluster to operate. Nothing here needs one.	
Cloud SQL / AlloyDB	No relational workload. Firestore is the database <i>and</i> the push channel; adding SQL would add a second source of truth.	
Vertex AI Vector Search	~\$65+/month of idle ScaNN infrastructure for ~10k vectors, where flat exact KNN is <i>more</i> accurate. Evolution path stated in 7.3.	
Transcoder API	Concat and trim only — no transitions, no text, no Ken Burns. It cannot make a reel.	
Agent Gateway	The one heavy GEAP component (network provisioning, 1–2 days). Registry/Identity would arrive with an Agent Runtime deployment this build deliberately does not make.	
Pub/Sub fan-out to workers	Pub/Sub has no server-side dispatch rate. The throttle <i>is</i> the architecture, so Cloud Tasks wins.	
Firestore triggers	Every event-driven surface here is Storage → Eventarc. A second event system for an occasional prose memo is an operational cost with no payoff; the tick already visits every live event.	
Dataflow / BigQuery	No streaming-analytics workload. Sharded Firestore counters feed the coverage ledger at O(1).	

2.5 THE EXPERIENCE LAYER

one codebase, four surfaces, zero polling

- A statically exported Next.js App Router PWA on Firebase Hosting — so the guest app is a CDN asset with no server rendering to keep alive, and the only backend a phone talks to is `api` plus Firestore's own listener channel.
- An IndexedDB outbox, not an upload form. Intent is registered first, URLs are cached against their expiry, and items left `uploading` by a closed tab are rewound at the start of the next drain. Closing the app mid-send is a normal event, not a lost photograph.
- A three-tier media ladder — memory, then an IndexedDB blob store with oldest-first eviction, then a signed URL — because an authenticated image cannot be fetched with a bearer token *and* follow a cross-origin redirect: a preflight cannot be redirected, and no bucket CORS policy can grant a *request* header. So the endpoint returns the URL as JSON and the browser loads it directly. **That was a real, product-wide defect, found on a phone rather than in an emulator.**

2.3 IDENTITY AND GRANT MATRIX					the "tools properly isolated and scoped" answer, in full
DEPLOYABLE	IDENTITY	SCALE	GRANTS HELD	NOTABLY <i>NOT</i> HELD	
api public	sa-api	0–n	datastore.user · cloudtasks.enqueueuer · firebaseauth.admin · aiplatform.user · modelarmor.user · signBlob on itself	no <i>bucket object</i> read on raw beyond signing	
intake	sa-intake	0–n	datastore.user · cloudtasks.enqueueuer · eventarc.eventReceiver · raw read · derived write	no GenAI grant at all — intake cannot call a model	
worker-curate ♦	sa-curate	0–n	datastore.user · aiplatform.user · derived read	no grant on the raw bucket — it classifies a thumbnail, not an original	
worker-face ▲	sa-face	0–n	datastore.user · derived read	no GenAI grant — identity never touches a language model	
worker-safety ♦	sa-safety	0–n	datastore.user · aiplatform.user · serviceUsageConsumer (Vision) · derived read	no raw grant	
worker-video-prep	sa-video-prep	0–n	datastore.user · cloudtasks.enqueueuer · derived r/w	no GenAI grant	
publisher	sa-publisher	1–5	datastore.user only	no bucket, no model. It ranks documents.	
dlq	sa-dlq	0–n	datastore.user only	touches no media, ever	
render job	sa-render	1 exec	datastore.user · aiplatform.user · modelarmor.user · derived read · curated objectCreator	no raw grant ; <code>objectCreator</code> not <code>objectAdmin</code> — a reel is immutable, a better cut is a new id	
Cloud Tasks OIDC	sa-tasks	—	serviceAccountUser on the four worker identities it may invoke	cannot invoke <code>api</code> as anything else	
Cloud Scheduler OIDC	sa-scheduler	—	the only principal on the <code>/internal/tick</code> allowlist	no data grants of its own	
Eventarc / Pub-Sub	sa-eventarc	—	eventarc.eventReceiver + token creation for push	no data grants	

The load-bearing consequence: the two services that read a guest's photograph have no permission to open the original, the service that runs the planner cannot read a photograph at all (the bounty validator is prompted with the Curator's *caption*, not the image — ~200 tokens instead of ~460, and no bucket read), and the only identity that can write a finished reel cannot overwrite one. **Signed URLs are minted through IAM signBlob on sa-api impersonating itself, which is why this repository contains no service-account key file and local development uses ADC.**

2.4 BUILD AND RELEASE

no local Docker on the build machine, by choice

- One image, fanned out.** `gcloud run deploy --source` hands the build to Cloud Build; the resulting tag is deployed only to the services whose changed modules are actually on their import path. A deploy note records which services were *deliberately* not redeployed — that is a decision, not an omission.
- Infrastructure is nine idempotent shell scripts** (`sa · buckets · queues · firestore · eventarc · scheduler · render · up · scale-down`). Re-running any of them is a no-op, so bootstrap and repair are the same command.
- The face worker's image is multi-stage** — the builder carries a toolchain, the runtime does not — and the ArcFace weights are baked in, so a cold start does not download a 326 MB model.
- Post-submission posture is scale *down*, not tear down** (`deploy/judge-mode.sh`): a slower director cadence, warm workers, and the demo event left standing, because a project that dies 48 h after the deadline fails the part of the judging window that runs for a month.

The data plane — a photograph's journey, and the state machine underneath it

Push everywhere, at-least-once everywhere, therefore **idempotent on a status transition rather than on a task id**. The stages complete in any order; status is derived, never assigned.

~6 s

BYTES TO THE WALL

5

PARALLEL STAGES

1

WRITER OF VISIBILITY

3.1 A PHOTOGRAPH'S JOURNEY				measured warm, on the live deployment
#	ACTOR	STEP	WHY IT IS SAFE	T
1	phone → api	POST /uploads: intent for N files + the consent ring for the batch	Intent is registered <i>before</i> bytes exist, so an abandoned upload is a known state, not a mystery object	—
2	api	N media docs at awaiting_upload; N signed V4 PUT URLs, 15 min, content-type and content-length pinned	A pinned length is the upload size limit — enforced by the storage layer, not by trusting a client	—
3	phone → GCS	PUT the bytes, 3 concurrent, resumed from an IndexedDB outbox if the app was closed	Bytes never transit a service, so a burst costs bucket throughput and no compute	—
4	Eventarc	object.v1.finalized → intake	At-least-once with a dead-letter after 5 attempts. Handlers are idempotent, so a redelivery is free	—
5	intake	size check · md5 dedupe · EXIF read <i>at the event's timezone</i> · lossless GPS strip · 3 derived renders · fan-out	Derived output goes to a different bucket , which is what prevents an ingestion retrigger loop	—
6	Cloud Tasks	3 unnamed tasks (photo) or 1 (video), dispatched at the queue's rate	Unnamed on purpose: a named task blocks its own replay for hours, and replay is the recovery path	—
7	worker-curate ♦	Curator verdict: stage, moments, aesthetic, caption, scene setting	Structured output; a schema-invalid answer twice is a <i>permanent</i> failure, not a retry storm	1.8 s
8	worker-safety ♦	SafeSearch hard gate, then the dignity rubric — and the rubric is skipped entirely when SafeSearch is decisive	A refusal writes host_review. There is no path from a model failure to public_ok	1.5 s
9	worker-face ▲	detect → align → 512-d embed → findNearest → link or cluster	Auto-linking is gated on claimApproved, so a pending claim accretes nothing while it waits	6–9 s
10	recompute_visibility •	Derives visibility and status and the coverage-shard bump in one transaction	One writer, whole-document read-modify-write: no race can leak a photo and no verdict can disagree with its exposure	—
11	publisher •	Rebuilds the kiosk program, debounced 1.5 s, written only if the decisions changed	The wall restarts its show on a new revision, so an unchanged rebuild must be <i>absent</i> , not merely idempotent	1.3–3.7 s
12	every client	Snapshot listeners repaint	Nothing polls. The just_in strip is a separate listener, so it is faster than the program	~1–2 s

End to end, warm: 3.6–4.4 s from uploadedAt to status=indexed, then 1.3–3.7 s to the wall — and about 42 s on the first upload after idle, because two workers sit at min-instances 0. That number is stated rather than hidden: it is the honest price of scaling to zero between events, and it is why the demo event pre-warms.

3.5 WHAT EACH PERCEPTION WORKER IS ALLOWED TO CONCLUDE			schema design is the guardrail
WORKER	OUTPUT, AS A CLOSED SCHEMA	STRUCTURALLY CANNOT	
Curator ♦	a stage id <i>from this event</i> · moment tags · aesthetic score · caption · sceneSetting from a nine-value enum · cultural elements filtered against a host-reviewed glossary	invent a stage · infer the stage from time of day or lighting · name a person · set any exposure field	
Guardian ♦	one of four verdicts — public_ok, private_only, host_review, blocked — bounded above by the host's declared sensitivity ceiling	make a verdict <i>less</i> conservative than SafeSearch already did · reach public_ok from any failure path	
Face Indexer ▲	face boxes, a 512-d unit vector, and a nearest-neighbour link or a new cluster	use a model to decide identity · link to an unapproved person · be read by any client	
Why the scene vocabulary is <i>closed</i> , and why that is an architecture decision rather than a lint rule: the accumulated distribution of those nine values <i>is</i> the world model. The count of indoor-venue against outdoor-nature across an event is what tells the system, without being told, whether it is in a ballroom or on a hillside — which is what lets a photograph of a valley be on-topic at a hill-station wedding and off-topic at a ballroom one. An open vocabulary makes that distribution uncountable (indoor, inside, hall and banquet_hall are four keys for one fact) and the arithmetic silently stops meaning anything. The signal was never in the pixels; it is in the distribution of the other photographs.			

3.2 THE MEDIA STATE MACHINE		parallel flags, derived status
Five stage flags, each pending → done failed failed_permanent, completing in any order . A photo takes thumb · curate · faces · safety; a video takes thumb · video_prep.		
STATUS	MEANS	
awaiting_upload	intent registered, no object yet	
uploaded	object finalized, not yet fanned out	
processing	at least one stage outstanding	
indexed	every stage reached done. Derived inside the visibility transaction, and the precondition every public-surface query filters on	
quarantined	a transient failure exhausted its attempts — we do not know what this is, so it stops and a human is told	
rejected / abandoned	host verdict · no object ever arrived	

KIND	STAGE	ITEM	ALERT
Transient, exhausted (rate limit, 5xx, timeout)	failed	quarantined	error
Permanent (refusal, or schema-invalid twice)	failed_permanent	unchanged — writes the stage's conservative default	warning
A permanent failure deliberately leaves the item alone. The guest keeps their own photograph in their own album and the event pool; because the item never reaches indexed, no public surface can ever serve it. Quarantining here would hide a guest's photo from the guest because a language model had an opinion — which is the exact failure the whole enforcement layer exists to prevent.			

3.4 BACKPRESSURE, WITH THE NUMBERS			
QUEUE	DISP/S	CONC	CONSUMER
classify-queue	8	10	worker-curate ♦
safety-queue	8	10	worker-safety ♦
face-queue	10	8	worker-face ▲
priority-queue	20	10	bounty submissions jump the line
video-prep-queue	2	2	worker-video-prep
renders-queue	1	2	configured; unused by the reel path — see 5.2
The rates are spend-tier arithmetic, not vibes. Two model queues at 8/s bound the sustained call rate onto the classifier tier, and the honest SLA follows from it: p50 ≤ 5 s at up to ~10 uploads/s; above that the queues drain FIFO, latency grows, and nothing fails — the wall simply shows the photographs that have cleared. Backpressure that is visible and bounded beats a system that pretends bursts do not happen.			

The control loop — one tick, six steps, and where the model stops being in charge

The autonomous half of the product. A Cloud Scheduler job takes a lease, aggregates evidence, asks one model one question, and then **re-derives every field of every proposed action from Firestore before executing it**. Nobody presses anything.

6

DETERMINISTIC
STEPS : 1 CALL

O(1)

LEDGER COST
IN CORPUS SIZE

10

TICK ROLLING
WINDOW

4.1

ONE TICK, IN ORDER — AND THE ORDER IS THE DESIGN

directors/story/director.py

1 · VALIDATE

Bounty submissions that reached `indexed` since the last tick. **Runs first** so the ledger already reflects the gaps the last tick's bounties closed — reasoning first would re-ask for a photograph we already have.

→

2 · EXPIRE

Anything past `expiresAt` becomes a **permanent coverage gap** on the director state, because the wrap report's honesty depends on remembering what was asked for and not delivered.

→

3 · ARM

If the stage changed, its required moments go live **from the timeline prior, with no model call**. A varmala lasts under a minute; reactive detection can only notice its absence once it is over.

→

4 · LEDGER

Six reads, whatever the event's size: coverage shards, open bounties, people, a 20-doc drift sample, two count aggregations. Nothing scans the media collection.

→

5 · REASON ♦

The **one** model call. `gemini-3.7-flash`, ADK `LLMAgent`, closed action vocabulary, Model Armor plugin in front. Only ids that already exist are printed into the prompt.

→

6 · ACT

Split into a **pure `decide()`** and a writer `apply()`, so the whole adversarial guardrail table runs offline with no network, no Firestore and no spend.

Two properties hold whatever happens inside. The tick **never takes a second lease** — it runs inside the one `api` already holds, and a director that took its own would deadlock against the lease protecting it. And the session write happens in a `finally`, so a deflected prompt, a rate limit or a schema-invalid answer still leaves the deterministic work committed and the window advanced: **a director that loses its memory whenever the model has a bad minute would re-arm the same bounties on the next pass**.

4.2

THE GUARDRAILS, WHICH ARE THE PRODUCT

directors/story/act.py

INVARIANT	ENFORCED HOW
≤ 2 new bounties per tick · ≤ 6 open at once	counted against the bounties actually open, inside the lease that serialises them
No duplicate per (stage, moment, person)	a dedupe key, re-derived rather than trusted
<code>points = clamp(base × vipWeight, 50, 300)</code>	<code>vipWeight</code> from the host-declared tier — deterministic metadata, never memory
Auto stage-advance	needs confidence ≥ 0.8 <i>and</i> the scheduled window to agree within 45 min <i>and</i> a target stage that has not lapsed
Every id in the plan	looked up in the stages, people and bounties that exist — a plausible-sounding invention is rejected with a logged reason
Who is in the photograph	a <code>personId</code> from a 512-d match. The model is never asked an identity question

4.3

TWO LEASES, TWO OPPOSITE DISCIPLINES

The tick lease is **mutual exclusion around an action**. Acquired at the top, released at the bottom; its 5-minute TTL is purely a crash backstop. Holding it for the TTL would *throttle* the cadence it exists to protect — a 900 ms tick would block the next four scheduled ones while looking healthy.

The publisher lease is **leadership over a resource**, held for as long as the process lives and renewed on a 45-second timer. Here expiry *is* the failover: an instance that is scaled away stops renewing and another takes the wall over inside the TTL. Verified live: one publisher instance held **nine** event leases concurrently and wrote nine playlists.

Same mechanism, one file. Conflating the two would have degraded a 30-second demo cadence to one tick every five minutes.

4.4

EVIDENCE THAT IT RAN UNPROMPTED

autonomy that cannot be checked is a claim

- platform/tickPulse** — one root heartbeat document per tick: `{lastTickAt, mode, caller, events, ticked, ms, ticks: Increment(1)}`, written *whether or not the tick found work*, because a tick that found zero live events is exactly as important to record. It is what an offline smoke test waits on to prove the Scheduler fired with nobody watching.
- A **cadence a cron cannot fake**. The demo event runs a 1-minute Scheduler job plus a +30 s Cloud Task interleave, so consecutive heartbeats land ~30 s apart — an interval a 1-minute cron cannot produce. Measured: `:06 → :36 → :08 → :38`.
- The **director's own reasoning is a document, not a dashboard** — assessments, deferred ideas and permanent gaps under `ledger/`, plus a structured log line per stage per item. The guest-facing countdown reads a narrow projection through the API rather than through a rules exception.
- A manual trigger exists, rate-limited and demo-only, taking the *same* lease so it cannot double-issue a bounty — and it is **labelled on screen as an override, with the real cadence counting down above it**. An unlabelled button beside a claim of autonomy contradicts your own video.

4.5

THREE STORES, THREE JOBS — AND MEMORY BANK IS NOT A CACHE

STORE	HOLDS	MUST NEVER HOLD
Firestore <code>system of record</code>	media state machine · verdicts · <code>visibility</code> · bounties · points · coverage ledger · VIP <code>tier</code> · director state	anything the agent merely <i>believes</i>
Rolling tick window <code>working memory</code>	the last 10 tick summaries: what was issued, deferred, assessed — so the model reasons over a narrative instead of a cold start	product state
Memory Bank <code>soft context</code>	the host's free-text standing preferences at <code>{eventId}:host</code> ; per-person taste memos	anything that gates exposure, spend, or who is important

Compaction is arithmetic, not summarization. Overflowing ticks fold into a *counted* sentence written by code. Asking a model to summarize its own history is how a long-running agent's context becomes a story about itself — every compaction is a lossy retelling of the last one, and after fifty ticks the numbers are vibes. **Counting cannot drift**.

The **performance instinct that reaches for a memory store is right; the mechanism is different**. Not re-scanning 5,000 photographs every two minutes is solved by a *materialized view inside the database* — sharded coverage counters bumped in the transaction that already sets `status='indexed'`. That cache is transactional, shardable and auditable. Memory Bank could not do this job: it is a network call, it has no transactions, and its contents are probabilistic.

Where the director runs, and why. On Cloud Run, inside the tick, *not* on Agent Runtime. Every guardrail above has to be serialised by the lease the tick already holds; a remote `:query` would move the reasoning and leave all of them behind — one extra network hop, two places to look, identical guarantees. **The importable `root_agent()` is still the single artifact** `adk deploy agent_engine` would package, so the door is open, not closed. The cost is a managed reasoning-chain dashboard, and the honest replacement is named in 4.4.

Stated plainly, because it is the kind of thing worth catching before a judge does: `AGENT_ENGINE_ID` is empty in production, so Memory Bank is *inactive* and the same host free-text is read off the event document. The integration is real code on a real API and it degrades to the document silently by design — but it is not running, and it is not counted as if it were.

4.6

THE AGENT CENSUS, COUNTED STRICTLY

the anti-agent-washing test

A component earns the word "agent" only if it makes context-dependent judgments, or plans and chooses actions. Under that test:

2 planning agents	Story Director · Reel Director. Goal, plan, tool use, self-correction loop.
2 perception agents	Curator · Guardian. Irreducibly contextual judgment, structured verdicts, no actuation by design .
1 specialized ML worker	Face Indexer. Deterministic embedding arithmetic, no LLM. Calling it an agent would be agent-washing.
Deliberate non-agents	intake · publisher · <code>recompute_visibility</code> · the render job · the itinerary parser (a one-shot extraction is a tool call, not an agent).

Eight LLM roles in total, counting the critic and the bounty validator. **The perception agents deliberately cannot act, and that separation is the entire trust architecture**.

The reel pipeline — evidence to a cut film, in one job execution

The second planning agent. A commission walks seven steps in a single Cloud Run Job, and the two hardest quality criteria — no face cropped by a camera move, no cut off the beat — are made **true by construction rather than tested for**.

7

STEPS, ONE
EXECUTION

5

PERSONA
LENSES

0.000

MS BEAT
ERROR

5.1

ONE COMMISSION, END TO END

SELECT

Diversity-sampled candidates from a **snapshot**, per persona: recap spreads chronologically across the arc, `main_character` narrows to one `personId`. Aesthetic floor with a candidate-count fallback.

→

DIRECT ♦

A narrative brief built from *actual* evidence — captions, moment tags, the stage diary, the world model — then a storyboard. Structural freedom inside a persona lens, never a template.

→

CRITIC ♦

A rubric pass over the plan, **capped at one retry**, behind a deterministic EDL linter. A loop that cannot terminate is not a self-improving agent.

→

SCORE

Arithmetic ranking of the accepted plan's shots.

→

EDL

Shot list quantized onto the beat grid; Ken Burns rectangles *derived* from face boxes.

→

RENDER

One ffmpeg filtergraph, 1080×1920, with the Lyria track and the cached Veo opener.

→

PUBLISH

The whole manifest is re-validated against current visibility, inside a transaction. A reel that *started* eligible can still be refused.

directors/reel/pipeline.py · resumable at stage granularity

Every step writes its output to the reel document as it lands and is skipped when that output is already present, so a commission that died in `render` does not pay for a second plan or a second Lyria clip — it re-reads them and re-renders. That is also what makes a retry safe to fire from a later tick. The document walks `directing` → `composing` → `rendering` → `published`, so when a reel does not appear there is exactly one place to look.

5.2

WHY ONE JOB, AND NOT A SERVICE IN FRONT OF A JOB

- **Cloud Tasks cannot start a Cloud Run Job.** Tasks speaks HTTP; a job has no URL and is started through the Run Admin API with per-execution container overrides. So the split costs an extra service, an extra identity and an extra hop — and the EDL has to be persisted on the reel document either way.
- **The consequence is stated rather than hidden:** `renders-queue`'s concurrency dial is therefore *not* the throttle it looks like. It could never express the real invariant anyway — "one active render *per persona, per event*", not a global number — so that invariant moved to a Firestore read inside the lease the Story Director already holds, with a staleness backstop because a job execution can die and leave its document in `rendering` forever.
- **Cost of the merge:** about forty seconds of an 8-vCPU job idling on model calls and the Lyria stream — under a cent, against a whole extra deployment surface.

5.4

THE CONSENT INTERLOCK IS NOT COSMETIC

- **A published reel is served by a redirect, never by a stored URL.** The reel document's `videoUri` is an `api` path that re-checks the reel's visibility on every request and 302s to a 60-minute signed URL. Not a public bucket — the object would stay fetchable after the reel is unpublished, which would make this whole interlock decorative. Not a stored signed URL either — it expires while the document still advertises it, which is a reel that plays on demo day and 404s during the judging month.
- **Playback is deliberately unauthenticated for a *published* reel**, on the same reasoning the kiosk playlist is world-readable: a `<video>` element cannot send an Authorization header, and a kiosk is a television in a venue. What makes that safe is that "published" is *derived* — only `recompute_visibility` and the publish transaction write it. **Downloading, by contrast, requires membership on every event: watching is not keeping.**
- **Retraction runs from the one writer of `visibility`**, and only on a public → non-public transition, because an `array_contains` query on every recompute would cost a query per stage completion per photograph and virtually all of those are pool → pool.
- **The renderer reads the same `display_1600` derived render the gallery serves** — so the zoom ceiling is 1.25, and a little softness is the price of not handing the reel renderer five hundred guests' full-resolution originals.

5.3

TWO QUALITY CRITERIA, MADE TRUE INSTEAD OF TESTED

No face crosses the frame edge. Asking the model for four floats per shot and linting them makes that a property we *test*. Asking it for a *gesture* — `PUSH_IN` — and deriving both rectangles from the detected face boxes makes it a property that **cannot be violated**, and the proof is three lines: a Ken Burns move is a linear interpolation between two axis-aligned rectangles, so each edge is linear in the interpolation parameter and attains its extremes at the endpoints; if both endpoint rectangles contain the faces' bounding box, every intermediate one does. The code guarantees the premise by walking the zoom back toward 1.0 until it holds — and when even an unzoomed 9:16 crop cannot hold a wide group line-up, the shot switches to a **fitted composition**: the whole photograph in frame over a blurred, drifting copy of itself. **A group photo is never cropped through somebody's head to make an aspect ratio work.**

No cut off the beat. The measurement is **0.000 ms**, not "within ±80 ms", because the EDL only ever emits values that are already on the librosa grid: the minimum and maximum shot-length clamps step to a *neighbouring beat* instead of applying an offset. Nothing ever leaves the grid to have to be snapped back onto it.

5.6

FIVE PERSONA LENSES, ONE PARAMETERIZED DIRECTOR

PERSONA	WHAT SELECT DOES DIFFERENTLY	COMMISSIONED BY
<code>couple</code>	the event's principals, weighted by tier	the director, or the host
<code>stage_recap</code>	bounded to one stage id, carried on the document	a stage ending
<code>guest_energy</code>	crowd and reaction frames rather than principals	the director
<code>main_character</code>	one <code>personId</code> ; the reel is written to audience ring 1 , never to the wall	on demand only — never fanned out per guest
<code>event_recap</code>	every stage, spread chronologically across the arc, with its own aesthetic floor and a candidate-count fallback	deterministically, by the first tick that sees wrapping

The target rides on the reel document, not in the enum, so **the persona stays a lens and the target stays data**. That is the difference between one parameterized agent and five copy-pasted ones — and it is why a persona/target mismatch is rejected at commission time rather than discovered at render time. Personas are also why the "one active render" invariant is *per persona, per event*: a recap and a stage cut are not competing for one slot.

5.5

WHAT A REEL COSTS, AND WHAT IT IS ALLOWED TO COST

LINE ITEM	MODEL / RESOURCE	COST
Plan + critic	gemini-3.7-flash ×2-3	< \$0.01
Soundtrack	lyria-3-clip-preview	\$0.04
Opener (once per event, cached)	veo-3.1-fast · 8 s	\$0.80
Render	8 vCPU job, single-digit minutes	~\$0.02

Volume rails, and they are ordinary host-settable configuration rather than a **demo-only branch** — a daily reel ceiling, a "commission at most one reel per N new media" counter, and per-day / lifetime media caps enforced *transactionally* in the same transaction as the per-guest upload rate limit. The discipline behind that: **a demo convenience is honest only if a real host could set it too**. A Lyria failure is explicitly *not* fatal — a silent reel is still a reel.

5 of 7 — the reel pipeline

one parameterized director, five persona lenses – not five copy-pasted agents

Trust & governance — judgment by agents, enforcement by policy

This system holds other people's photographs and other people's faces. So the exposure decision is a **single deterministic transactional function**, the biometric lives in a collection with no read rule at all, and **no face match grants anything on its own**.

107

RULES
ASSERTIONS

0

LLM WRITES OF
VISIBILITY

0

CLIENTS THAT CAN
READ A FACE

6.1

THE EXPOSURE DECISION, IN FULL

shared/visibility.py — the only writer

Six inputs, one output, no model anywhere near it. Missing inputs are not an error: a half-processed item is `pool`, never `public`.

CONDITION	RESULT
deleted, or the host's panic freeze is on	self
Guardian verdict <code>blocked</code>	self
consent ring 0 (self-only)	self
a subject in the frame has vetoed	pool
ring 2, but verdict $\neq$ <code>public_ok</code> or aesthetic < the event's public floor	pool
ring 2, <code>public_ok</code> , above the floor, no veto	public
otherwise	pool

Three things happen in that one transaction: `visibility` is written, `status='indexed'` is derived (it requires *every* stage `done`), and the coverage shards are bumped. There is never a moment where a verdict and its exposure disagree, and the ledger can never drift from the corpus it summarizes. The acceptance test is a `grep`: exactly one writer of that field.

6.2

CONSENT IS THREE RINGS, AND RING 1 IS THE DEFAULT

0 • self	Only the uploader. Not even the people in the photograph.
1 • pool	The default. The event's members, and the private album of every person detected in it.
2 • public	Opt-in <i>per batch</i> , and still subject to every gate in 6.1. Nothing is public by accident.

Retroactive within seconds, plus a subject veto that works on a photograph you did not upload, and a real per-subject deletion path that removes the person document and the stored face template — not a flag. A denial of a claim **reverses** rather than records: it revokes the claim, and deletes the person and the template when that claim is what created them, because a stored template left behind is an unapproved biometric sitting in the match index.

6.3

MODEL ARMOR, IN TWO PLACES, BECAUSE THEY FAIL DIFFERENTLY

- At the surface that accepts the text (the host's pasted itinerary, the bounty brief) — precise about what it rejected, and able to tell a human.
- As an ADK plugin in front of the director models — which is what catches text that reached a prompt by *any* route: a tool result, a recalled memory, a caption threaded into a brief three hops from where it was typed. The surface check alone would be a list of places to remember.
- Deliberately not attached to the perception workers. A per-photo Armor call would add a network round trip to a stage whose queue rate is calibrated, in order to inspect text the host already had checked at onboarding.
- A deflection is a **permanent** failure by construction, so it takes the conservative path every worker already has — no retry storm on text that will match again.

6.7

THE RULE THAT GOVERNS EVERY DEMO AFFORDANCE

A demo convenience is honest if it is a configuration value a real host could also set. It is a thumb on the scale if it is a code branch keyed on "this event belongs to a judge."

Under that rule the guided tour costs nothing in trust: the page, the seeded event, one-tap join, the pre-warm, the compressed timeline and the faster tick cadence are all things the product does for anyone. **Two things failed the rule and were deleted rather than defended:**

- A demo-only override of the public aesthetic floor, gated on the event's class. It was a second path to a value the ordinary host-settable field already provided — and it was **the only exposure logic in the system whose condition was whose event it is**. Deleted; the demo event now sets the plain field any host can set.
- A flag that auto-promoted a self-enrolling guest to VIP. A genuine branch no real host could trigger, existing only to make a visitor feel featured. It ships off.

There is no route, component, flag or query parameter named "judge mode" anywhere in the code — the label itself manufactures the suspicion that a special path exists for being looked at. The ranking-explanation overlay is `?explain=1`, and it only reveals numbers the publisher already stored: it changes no query, no

6.4

RULES GRANT WHOLE DOCUMENTS, SO THE DATA MODEL HAD TO MOVE

Firestore has no field-level read control. `people/{personId}` *must* be readable by every event member — the kiosk's uploader credit, the leaderboard's names, and the deterministic `tier` $\rightarrow$ `vipweight` ranking all read it — and that same document was holding the face template. **Any rule permitting the first would have handed every guest a copy of everyone's face.**

- The biometric moved to `events/{id}/enrollments/{personId}`: a **collection with no `allow` rule at all** — unreadable by every client *including the host*, because a console has no use for an embedding and "the host can see it" is how a biometric ends up in a screenshot.
- `people/{id}/private/*` is reserved and deny-all for anything else genuinely personal — the Gemma-written taste memo lands there, not on the person document.
- The read boundary is the event, not the sign-in.** `isMember(eventId)` resolves a `members[]` custom claim minted by `POST /join`. It replaced a predicate that was literally "is signed in", which let anyone holding any event id read that event's wall, guest list, names and tiers. Membership rides in a claim rather than a document because **a rule may not perform a `get()`**.
- Query shape and rule shape are one design.** Firestore fails an *entire* query when one returned document is denied rather than skipping it — so every listener carries the filters its rule requires, and the matrix runs those exact queries as those exact personas, **including rows that assert dropping a filter correctly breaks**.

6.5

THE HOST APPROVES EVERY ALBUM — NO EXCEPTIONS TO ENUMERATE

A live defect closed by **deleting the automatic grant rather than narrowing it**: enrolling with a selfie writes a *held* claim and grants nothing — no identity claim, no uid link, no `personId` on any face. Identity is granted in exactly two places: a host approval, and a magic link the album's own owner minted (where possession is the proof).

- The face worker's auto-linker is gated on `claimApproved`, so a *pending* enrolment cannot quietly accrete an album while it waits for review — otherwise the photographs handed over would already be in a stranger's album by the time the host looked at the card.
- A VIP match that is also ambiguous is always labelled `protected_person`, never `ambiguous_match`: the more actionable signal wins on the review card.
- The enrolment path carries the same ban check and hourly cap as uploads, on **its own window fields**, so a photo burst and a selfie probe cannot hide inside each other.
- The cost is one host tap per guest, which is what a real event looks like.

6.6

WHAT THIS BUILD DOES **NOT** CLAIM

stated here so it cannot be discovered instead

Agent Runtime, Identity, Registry, Observability	Zero deployments. Nothing is wired to them, so the rows are deleted rather than softened. Memory Bank is wired but inactive (<code>AGENT_ENGINE_ID</code> empty).
OpenTelemetry span DAGs	Not instrumented. The evidence of a tick is a structured one-line-per-stage log, a heartbeat document, and the director state — named on page 7.
Reel v2 supersession, collages	Specified, not built. One version per commission.
Data purge / retention job	Does not exist , and an earlier claim that it did was retracted. The working deletion path is per-subject and on request.
Two residual read exposures	A member reads whole media documents; the kiosk playlist is world-readable because narrowing it would require a <code>get()</code> . Documented as limits, not closed.
InsightFace weights	Non-commercial research licence — fine here, and the production swap (AuraFace / SFace) is named.

Failure, scale & cost — duration is the premise, fault tolerance is the proof

A two-second task does not need a dead-letter queue. Long-running is not a virtue on its own — it is the *reason* failure tolerance has to be designed. Here is every way this system is known to break, and what happens when it does.

\$10-40

REALISTIC COST PER EVENT

1

CHAOS SWITCH, CLASS-GATED

0

ERROR LOGS SINCE THE LAST DEPLOY

7.1 FAILURE AND RECOVERY MATRIX			
		every row has a code path, not an intention	
FAILURE	DETECTED BY	RESPONSE	BLAST RADIUS
Model rate limit or 5xx	status classified at the one model wrapper	Cloud Tasks backoff, up to 5 attempts	one item, delayed
Model refuses, or returns schema-invalid output twice	the same wrapper, as permanent	write the stage's conservative default; item keeps its album; warning	none — it never reaches a public surface
Transient failure exhausts its attempts	attempt counter	stage failed, item quarantined, error alert	one item, and a human is told
Poisoned event, 5 Eventarc attempts	Pub/Sub dead-letter subscription	dlq service quarantines and alerts	one item; the trigger stays healthy
A director hallucinates a stage, a person or a bounty that does not exist	ACT re-derives every field from Firestore	action rejected with a logged reason; the rest of the plan still applies	none
A critic loop will not converge	retry counter + a deterministic EDL linter	capped at one retry, then the linted plan is used	one reel, slightly worse
Two ticks overlap; a tick replays	transactional lease + per-tickId idempotent recording	the second is a no-op; no double award is possible	none
A publisher instance is scaled away mid-lease	renewal stops; the lease expires	another instance takes that event's wall over inside the TTL	one wall, one TTL stale
A render job execution dies in rendering	staleness backstop inside the tick	re-commissioned; completed steps are re-read, not re-paid for	one reel, delayed
Lyria unavailable	exception at the music step	a silent reel is still a reel	one soundtrack
FCM outage	send never raises	the bounty still lands via the Firestore listener banner	one notification
Prompt injection in host-pasted text	Model Armor, at the surface and at the model	permanent failure, conservative path, zero tokens spent	none
A stage pin goes stale	a pin is honoured only until its own window ends, plus a grace	the schedule resumes automatically	none — this was a real defect, found and closed

Failure is exercised, not asserted. A class-gated chaos switch injects a transactional, decrementing 503 into a worker, and the recovery is watched end to end: 503 → backoff → attempt 2 → done. Synthetic dead-letter delivery has been driven through the DLQ path. The adversarial guardrail table for the director runs offline — no network, no Firestore, no spend — because **a rule that can only be exercised by coaxing a live model into misbehaving is a rule nobody checks.**

7.5 OBSERVABILITY, STATED HONESTLY	
what exists, and what does not	
There is no OpenTelemetry instrumentation and no Agent Observability dashboard in this build, because there is no Agent Runtime deployment to attach one to (4.5). Claiming a reasoning-chain span DAG would be the easiest thing on this page for a judge to falsify, so here is what exists instead — all of it queryable, and none of it a screenshot of a product page:	
QUESTION	ANSWER, AND WHERE IT LIVES
Is the fleet acting on its own?	platform/tickPulse — a rising counter and a fresh lastTickAt, written by every tick
What did the director decide, and why?	ledger/directorState — assessments, the rolling window, deferred commissions, permanent gaps
What did it reject, and why?	a structured log line per rejected action, carrying the guardrail that refused it
Where did this photograph spend its time?	stageTimings on the media document, plus one log line per stage with ms
What has this event cost?	derived on read from what was actually spent — never an incremented counter that can drift
Did anything break?	the ops/ collection, sharded per worker type so no document is hot, with a severity on every alert
The trade is stated rather than buried: what a managed runtime would have granted automatically is a span DAG and a token dashboard. What running the director inside the tick buys instead is that every number which decides who gets paid lives in one file and can be checked offline, with no network and no spend. Given the choice between an observability tab and an auditable guardrail, this build took the guardrail.	

7.2 WHAT BREAKS AT 10× — AND THE STATED EVOLUTION PATH		
DIMENSION	TODAY	FIRST THING TO BREAK, AND THE MOVE
Faces	~10k vectors, flat exact KNN	Exact KNN stays more accurate here and costs nothing idle. At ~1M faces, move to a managed ANN index — that is a swap of one query, because the embedding contract does not change.
Ingest burst	p50 ≤ 5 s to ~10 uploads/s	Beyond that the queues drain FIFO and latency grows; nothing fails. The dial is the queue rate, and raising it is a spend decision, which is why it lives in configuration.
Coverage ledger	6 reads, O(1) in corpus size	Already sharded per stage. The next limit is shard write contention, not read cost.
Hot documents	ops telemetry sharded per worker type; spend derived, never incremented	An Increment on the event document would have put 16 writes/s on one key against a ~1/s ceiling — every photo contending with every other. Derivation avoids the hot key entirely.
Concurrent live events	a transactional platform cap, enforced server-side	Per-event leases mean adding events adds parallelism, not contention: one publisher instance has held nine concurrently.
Small events	the control loop is evidence-driven	At five guests there are no statistical gaps, so the director correctly issues nothing. The system degrades by event size, and that is a property of being evidence-driven rather than template-driven.

7.3 COST, AND THE RAILS THAT BOUND IT		
LINE	BASIS	PER EVENT
Perception	flash-lite on thumbnails, not originals	< \$5
Director reasoning	one call per tick; idle ticks cost zero	< \$1
Reels	\$0.04 Lyria + \$0.80 cached opener	\$1–3
Storage, Firestore, Run	free tiers cover a multi-day event	< \$6
Total	a full multi-day event, end to end	\$10–40
The rails are code, not intentions: queue rates as the sustained call cap · costSoFarUsd derived on read so the host console's number is never confidently wrong · per-day and lifetime media caps enforced in the same transaction as the per-guest rate limit · a reel-per-N-media counter · an hourly sweep that wraps a lapsed public event and enforces its ceiling · a platform kill switch · budgets.		

- 7.4 HOW TO FALSIFY THIS DOCUMENT
- gcloud run services list returns the eight services on page 2, plus one job.
 - gcloud scheduler jobs list returns three ENABLED jobs at the stated schedules.
 - platform/tickPulse has a lastTickAt inside the last two minutes and a rising counter — the cheapest single check that the fleet is acting on its own.
 - make rules-test prints 107 assertions across 8 groups, with no cloud credentials.
 - make eval grades 25 golden fixtures over 169 checks.
 - A saved, event-filtered Logs Explorer query streams stage=... media=... ms=... verdict=..., one line per stage per item.
 - Every box on page 1 maps to something deployed. Anything specified but not built is on page 6, section 6.6, instead.